

SOR, MULTIGRID AND PRECONDITIONED
CONJUGATE GRADIENTS FOR SOLVING
A DIFFUSION PROBLEM ON THE
CYBER 205

by

John Gary
Steve McCormick
Roland Sweet

Scientific Computing Division
National Bureau of Standards
Boulder, Colorado

ABSTRACT

The purpose of this paper is the treatment of three numerical algorithms [successive overrelaxation (SOR), multigrid (MG) and conjugate gradients preconditioned by a fast Poisson solver (CG)] for solving large but mildly behaved diffusion problems on the CYBER 205 vector computer. The problem is a symmetric nonnegative definite matrix equation arising from a cell-centered finite difference approximation of a 3-d diffusion equation with full Neumann boundary conditions.

1. Introduction

The purpose of this paper is to report on the study of three algorithms [successive overrelaxation (SOR), multigrid (MG) and preconditioned conjugate gradients (CG)] for solving diffusion problems on the CYBER 205 vector computer. We have in mind the comparative study of each method as it applies to a time-dependent problem that simulates the flow of heat in a room due to a flame. The software package that we consider for this purpose is the so-called NBS fire model developed for the CRAY 1 by R. G. Rehm and H. R. Baum (cf. [RE]) and modified for the CYBER 205 by the authors with assistance from staff members of the Institute for Computational Studies at Colorado State University. The study of SOR, MG and CG will focus on their potential for efficient solution of the singular pressure equation that must be solved at each time step used in this model.

The next section describes the problem used for our test; Sections 3, 4 and 5 describe SOR, MG and CG, respectively; Section 6 describes the results of numerical tests with MG and CG; and conclusions are given in Section 7.

2. Problem Description

The matrix problem that arises at each time step in the fire model results from a spatial discretization of a 3-d pressure equation that we write as

$$(2.1) \quad \begin{aligned} -\nabla(\sigma \nabla U) &= f && \text{in } \Omega \\ U_n &= 0 && \text{in } \partial\Omega. \end{aligned}$$

Here, U_n represents the normal derivative, Ω is the unit rectangle in $\mathbb{R}^3$, and σ is a real-valued (diffusion) function defined on Ω .

The discretization of (2.1) is by symmetric central finite differences using a seven-point stencil on a cell-centered grid with uniform spacing in each coordinate direction. For simplicity of coding, we restrict our attention to the unit cube with $h = \frac{1}{32}$ in each direction, leading to $32^3 = 32,768$ total cells. We write the discrete problem as

$$(2.2) \quad L^h U^h = f^h.$$

Because of the uniform discretization and mildly varying behavior of for our application, we assume that the nonzero entries of L^h are positive and mildly varying. More precisely, we will treat (2.2) as if it is "nearly" a Poisson problem. The implication is in our choice of methods: SOR, which usually degrades with increasingly variable coefficients; MG, which uses the simplest of basic processes (e.g., piecewise constant interpolation); and CG, which is based on a fast Poisson solver as a preconditioner.

3. Red-Black SOR

A "best" case convergence rate for a finely tuned SOR algorithm applied to diffusion equations discretized on a 32^3 grid may be estimated from known theoretical results for Poisson's equation. The cost of a fully vectorized version of this algorithm can then be estimated in several ways. Our approach was to implement a simple red-black version and use its timing as a lower bound. These two considerations provide a liberal estimate for the potential performance of an SOR routine for our application. This estimate compelled us to conclude that SOR would not be competitive with either of the other two algorithms treated here, so we devoted no further effort in this direction. Nevertheless, the basic SOR algorithm was relatively simple to code and it illustrates the essential ingredients of the relaxation process used in a version of multigrid. We describe the implementation of red-black SOR in this section.

To permit effective use of the vector capabilities of the CYBER 205, the mesh is geometrically decomposed into "red" and "black" cells as depicted in Figure 1. Then one red-black SOR sweep consists of an update of the approximation at all the red cells followed by an update at the black ones. Since L^h is based on a seven-point operator stencil, then the red (black) cells are mutually decoupled. Thus, red (black) cell relaxation is nonrecursive and therefore fully vectorizable.

Red-black relaxation can be implemented in two basically different ways, depending on the choice of data structures. A lexicographic structure, where the unknowns are ordered without regard to their "color", is

perhaps the easiest to code. A red-black data structure, where the unknowns are ordered with red cells first (but lexicographically within the red cells), is somewhat more cumbersome to implement but ultimately much more efficient.

To see this in more detail, note that the data structure for L^h consists of four "diagonal" vectors D1, D2, D3 and D4 which have the same ordering as L^h . Here, D1 contains the central coefficient for the stencil of L^h , D2 contains the coefficient pointing to the nearest neighbor in the positive x-direction, D3 in the positive y-direction, and D4 in the positive z-direction. D2 contains a zero at each right boundary point and similarly does D3 and D4. Thus, disregarding boundary points, a matrix multiply $v^h = L^h u^h$ can be implemented by performing operations of the form

$$(3.1) \quad v \leftarrow v + D_* \cdot u_*$$

where D_* and u_* are possibly shifted entries of D and u, D representing one of the diagonals D1, D2, D3 or D4. Seven operations of this type form the basic SOR relaxation process. (The operation with D1 is actually of the form $v \leftarrow u/D1$.)

The CYBER 205 vector operations require that the data lie contiguously in memory (see Appendix A). Thus, for the lexicographic data structure, the operations of the type in (3.1) must be performed on all of the points at once. That is, a red cell relaxation should be implemented by performing a full Jacobi-type sweep where all of the necessary operations are done first and this is followed by a "controlled store" correction to the red points only (see Appendix A). Thus, after all of the operations of the form in (3.1) are performed for *each* entry of u, then only those entries of v corresponding to the red cells are used to update the corresponding values of u.

Note that a lexicographic data structure allows for operations of the form in (3.1) to apply to the full length of v since we have supplied zeros in the proper boundary locations of each D. Note, however, that a red-black sweep involves a relaxation sweep over all cells *twice* so that relaxation is nearly twice as costly as one would expect.

A red-black data structure (where u is stored in red, UR, and black, UB, arrays) complicates coding but provides for greater efficiency since the red (black) sweeps may be done on the red (black) points only. The complication is that the black neighbors of red points are in a fixed relationship (and vice versa) only if the number of cells in each coordinate direction is odd. For example, in the odd case, the neighbor of UR(I) in the positive x-direction is simply UB(I).

When the number of cells in each direction is even, such as for the multigrid algorithm described in Section 4, an artificial cell may be added at each of the positive x and y boundaries to make the number odd in these directions. A fully vectorized red-black relaxation using a red-black structure can then be performed on all $33^2 \times 32$ cells by ignoring the results at the artificial cells. Note that this requires a modest increase in the total number of cells for our size problem by the factor

$$33^2 \times 32/32^3 \approx 1.06.$$

Note that this requires an expand operation on the approximation and right-hand side before relaxation and a subsequent compress of the new approximation.

Instead of adding artificial cells for each half relaxation sweep, we could perform a scatter operation in turn on the neighbors of opposite color in the positive *and* negative x-directions. (The y and z neighbors of UR and UB are in a fixed relationship for the even as well as the odd case, so the transformation via the scatter operation is needed only for the x neighbors.) However, this is less efficient than the artificial cell scheme we use in MG.

To see this, assume that the CYBER 205 is a two-pipe system so that the approximate times of the relevant vector operations (ignoring start up, which is reasonable for our application) is given by

<u>Operation</u>	<u>Time (nsecs)</u>
Multiply/Add	10M
Divide	17M
Scatter	25M
Compress/Expand	10M

where M is the length of the input vector (Actually, the compress/expand times are slightly larger since they really depend on the length of the result vector.) Since there are six adds, six multiplies and one divide per cell per relaxation sweep, the ratio of times for the artificial cell approach and scatter approach (ignoring the cost of compress/expand for the former) is just

$$\frac{13.7 \times 1.06}{18.7} \approx 0.78.$$

Even when the compress/expand operations are used an average of, say, every two full relaxation sweeps (as for MGW in Section 4), this ratio is

$$\frac{13.7 \times 1.06 \times 2 + 3}{18.7 \times 2} = 0.82.$$

The artificial cell approach we have adopted is clearly the most efficient.

The implications that these structures have on the multigrid algorithm discussed in the next section.

4. Multigrid

4.1 Basic Processes. For convenience and brevity, we will adopt the notation, and assume that the reader is familiar with the basic ideas, of [BR]. The essential processes that our algorithm incorporates are as follows:

- 4.1.a. Interpolation. Linear interpolation is a popular choice for problems of the type we are treating. However, because each coarse grid is constructed from the next finer one by deleting every other grid line (see Figure 1), then implementation of the process on the CYBER 205 can be fairly complicated. Not only does each coarse grid cell contribute to 64 fine grid cells (32 of which are black), but special care must be taken at the boundaries. On the other hand, (piecewise) constant interpolation (see Figure 2) is relatively simple since each coarse grid cell affects precisely those eight fine grid cells (four of which are black)

that represent the same geometric region. (We refer to these as the *corresponding* fine grid cells.) Thus, not only are the operations significantly reduced (by about a factor of eight), but they are more vectorizable since this correspondence is very regular. Moreover, although constant interpolation places a greater burden on relaxation, tests show that careful implementation can lead to a very effective and efficient algorithm. In fact, constant interpolation of the correction to the black points (which is all that is needed for seven-point fine grid operators and red-black relaxation) followed by a red cell relaxation sweep is very similar in complexity and speed to linear interpolation and correction to the red points (see the discussion in Section 4.3.a.). Moreover, constant interpolation together with the other multigrid processes satisfies certain variational operator relationships that are useful in theory and practice (see Section 4.1.c below), so it is our choice for I_{2h}^h in the present algorithm.

4.1.b. Fine-to-coarse transfer. We chose I_h^{2h} as the natural averaging operator depicted in Figure 2 and given by

$$(4.1) \quad I_h^{2h} = \frac{1}{16} I_h^{hT}.$$

That is, I_h^{2h} assigns to each coarse grid cell $\frac{1}{16}$ of the sum of the values of each of its corresponding red fine grid cells. (Black cells will never be used in I_h^{2h} since it will only be applied to the residual immediately after a black cell relaxation sweep.)

4.1.c. Coarse-grid operators. L^{2h} is just the natural symmetric seven-point finite difference operator analogous to L^h . Note that

$$(4.2) \quad L^{2h} = I_h^{2h} L^h I_{2h}^h.$$

Thus, the multigrid operators satisfy the so-called [MC]

variational conditions given by (4.1) and (4.2), an aid in analysis and an enhancement to practical application. This is because the energy functional now provides a very sharp error analysis tool to monitor performance and because each stage of multigrid now monotonically reduces this functional.

- 4.1.d. Relaxation. Constant interpolation requires that enough relaxation sweeps be done to smooth the oscillatory errors introduced by each coarse grid correction. Too few sweeps (e.g., no more than one before transfer of the residuals to the coarser grid and one after coarse grid correction) can lead to a divergent multigrid process. Our tests have actually shown that a very effective approach is to perform only a black cell sweep before coarse grid transfer and two red-black sweeps followed by a red cell sweep after coarse grid correction. (We refer to this as B-RBRBR relaxation.) Further relaxation steps before residual transfer prove to be unwarranted, although an additional full sweep after the correction can prove useful. We think the best general scheme is B-RBRBR, however.
- 4.1.e. Singularity. Because the variational conditions are satisfied, singularity of L^h due to full Neumann boundary conditions causes no difficulty. This follows because the null spaces of the symmetric operators L^{2h} and L^h are of the same type (i.e., constants) and because I_{2h}^h maps the one to the other.
- 4.1.f. Cycling details. Our algorithm is a fixed (as opposed to accomodative [BR]) correction scheme process using W-cycles, which are much more efficient here yet only slightly more expensive than V-cycles. We did not implement FMG [BR] since cubic interpolation would be very complicated and, with lower order interpolation for

our application, there is very little difference between a full W-cycle and an FMG that accounts for good initial fine grid approximations (which our motivating time-dependent problem provides).

4.2 Implementation. Appendix B contains a listing of our MGv routine together with a test driver main program. It is a simplified version that uses a basic lexicographic data structure (see Section 3) and implements red-black Gauss-Seidel relaxation by performing Jacobi iteration *twice*, once with controlled store of the correction to the red cells and once more for the black. Another version, which we refer to as MGv1, uses the same basic data structure except in relaxation where it translates to red-black storage. It is substantially more complex because it requires pointers and control vectors for translating between each of the data structures. MGv2, which is a version that uses red-black as the basic data structure, is even more complex. We thus focus our attention on MGv except to make a few comments on the other versions and to present the results of timing tests in Section 6.

4.2.a. Input parameters. Input to the MGv subroutine are vectors U and F each of dimension NX by NY by NZ that represent the present approximation and right-hand side, respectively; the grid points are ordered lexicographically. The seven-point operator stencil, which is assumed to be symmetric, is stored in the vectors D1, D2, D3 and D4, each of which are of dimension NX by NY by NZ. Here D1(I, J, K) is the central entry of the stencil and D2(I, J, K), D3(I, J, K) and D4(I, J, K) are the stencil entries pointing to the nearest neighbors in the positive x, y and z directions, respectively. It is assumed that these entries are zero at appropriate boundary points (e.g., $D2(NX, J, K) = 0$). Other input parameters are the number of grids, NG (which is restricted so that 2^{NG} is a common factor of NX, NY and NZ), the cycling control parameter, IGAMA (IGAMA = 2 specifies the W-cycle and IGAMA = 1 the V-cycle), the number of cycles, NCYC, and a parameter

ICALL that tells MGv if it has already been called for this problem (i.e., if ICALL > 0 it assumes that the various array pointers have already been determined). Finally, MSH and MSHC are used via a parameter statement to properly dimension work arrays.

4.2.b. Output. The only output in this version is the updated approximation which is contained in U. However, the subroutine RSDUAL can be used - preferably in RELAX - to output the Euclidean norm of the residual at various stages of the algorithm. This will considerably affect timing because each call requires computation of the residual. An alternative is to (conservatively) estimate the residual by computing the norm of the residual computed in the last half step of relaxation.

4.2.c. Set-up. The routine INDXS sets up pointers and bit vectors (see Appendix A) used in implementing the various multigrid processes described below. The cost of this routine, which is small in any case, is amortized over the number of calls made to MGv since INDXS is called only when ICALL = 0.

CGCOEF sets up the lexicographic data arrays needed by MGv for each grid level. This routine must be called each time MGv is called since the presumption is that the diffusion coefficients are changing. (CGCOEF must be modified for MGv1 to set up the red-black data structures and pointers for data translation between these types of structures.)

4.2.d. Relaxation. MGv execution time is dominated by RELAX due mainly to the use of constant interpolation and the way we have implemented red-black relaxation. This implementation is by way of a controlled store of a full Jacobi correction to the red cells followed by the same to the black. This allows MGv to maintain a basic lexicographic data structure that is convenient

for vectorization of the other processes, although at a cost of nearly a factor of two in performing relaxation.

(MGV1 uses a compress operation at the beginning of each call to RELAX to convert from lexicographic to red-black data structure. This is followed by the appropriate number of red-black relaxations, and an expand back to lexicographic ordering.)

- 4.2.e. Fine-to-coarse residual transfer. In MGV, to accommodate lexicographic ordering, RESCAL computes the full residual disregarding the fact that the black cell residual is zero. To transfer the red cell residuals to the coarse grid, a pointer IRESR (see Figure 3) is used that indicates for each coarse grid cell the location of the corresponding fine grid cell with smallest x, y, z coordinates. This fine grid cell is necessarily red so that a simple gather operation (see Appendix A) using IRESR accounts for the contribution of these points to the coarse grid. The contribution from each of the other three types of red points can be accounted for by constant offsets of IRESR. Thus, the residual is transferred to the coarse grid by summing the results of four gathers and dividing by 16.
- 4.2.f. Interpolation. INTAD performs constant interpolation of the coarse grid correction to the fine grid and adds it to the approximation there. Since this is always followed first by red cell relaxation which would overwrite any red cell correction, INTAD corrects only the fine grid black squares. This is done simply by four scatter operations that assign coarse grid values to their corresponding fine grid black cells, each defined by constant offsets of IRESR.

4.3 Potential Improvements. MGV, MGV1 and especially MGV2 are very efficient for the applications we consider here as evidenced by the timing and performance results reported in Section 6. Nevertheless, there are several aspects of these schemes that may be improved. Four important possibilities which we have yet to implement are:

- 4.3.a. Linear interpolation. Although linear interpolation would increase code and computational complexity, it is important to study the trade-offs here in terms of increased efficiency. We have not yet developed a vectorized version, but we estimated its complexity and tested its potential performance by making several experiments with a scalar version. The linear MG routine used B-RBR relaxation which, when properly vectorized, should be only slightly faster (by a half-relaxation sweep per cycle) than MGV2. We therefore compared these two routines over a range of several parameters: V- to W-cycles, smooth to oscillatory initial errors, Poisson to variable coefficient operators, and one to five cycles. In each case, MGV2 produced smaller residual errors, although the difference was significant only for smooth initial errors. There may therefore be at best only marginal improvements to be gained by converting INTAD to linear interpolation.
- 4.3.b. In-line subroutines. Replacing the subroutines by in-line code should lead to a substantial savings in time by eliminating the overhead in subroutine calls. The savings reported in [BA] may be quite different for our code because it is RELAX-bound (by virtue of constant interpolation and the 3-d geometry) on the one hand but uses W-cycles (with consequently greater subroutine calls) on the other.
- 4.3.c. Modified grid structure. The code complexity and efficiency of MGV2 can be improved somewhat by changing the input restriction on the number of cells in each

coordinate direction and restructuring the basic process. Thus (instead of NX, NY and NZ), we would require NX + 1, NY + 1 and NZ + 1 to be divisible by 2^{NG} . This would eliminate the need for the artificial points and the attendant overhead in RELAX provided we could maintain this oddness on coarser grids. This could be done by the so-called expanded boundary approach [H0] where, in essence, the positive-most cell in each coordinate direction is viewed (for intergrid transfer purposes only) as a full-sized coarse grid cell (i.e., twice the fine grid size in each coordinate).

- 4.3.d. Improved MGV1. Although MGV1 is less efficient than MGV2 (see Section 6), this 16% gap can be narrowed (by perhaps 7%) by computing the residual needed in RESCAL before the expand operation in RELAX.

5. Preconditioned Conjugate Gradients

In this section we describe the implementation of a preconditioned conjugate gradient algorithm that has been successfully used to solve equation (2.2) in a model run on the CRAY-1. The specific form of σ in (2.2) for the fire model is

$$\sigma = \frac{1}{1 + \zeta}$$

where $\zeta = \rho(x, y, z, t)$ is a relatively smooth perturbation satisfying $|\rho| < 0.3$.

Hence, it has been determined that a fast Poisson solver (FPS) is a very effective preconditioner for the conjugate gradient algorithm.

A complete description of this algorithm for the two-dimensional equation can be found in [LE]. For completeness, we write out the algorithm applied to equation (2.2) (with the superscript h deleted):

- Given an initial guess U_1 , form

$$v_1 = LU_1 \quad \text{and} \quad r_1 = f - LU_1,$$
 solve $Mu_1 = r_1$ and set $p_1 = u_1$.

- For $i = 1, 2, 3, \dots$ until convergence

$$\text{form } v_i = Lp_i,$$

$$\text{set } \alpha_i = r_i^T u_i / p_i^T v_i,$$

$$\text{update } U_{i+1} = U_i + \alpha_i p_i,$$

$$r_{i+1} = r_i - \alpha_i v_i,$$

$$\text{solve } Mu_{i+1} = r_{i+1}$$

$$\text{set } \beta_i = r_{i+1}^T u_{i+1} / r_i^T u_i,$$

$$\text{update } p_{i+1} = u_{i+1} + \beta_i p_i.$$

The matrix M in this algorithm represents the approximately scaled finite difference approximation to the three-dimensional Poisson operator with Neumann boundary conditions prescribed on all boundary faces of the parallelepiped. (In fact, M arises from L^h with $\sigma = 1$.)

Our task is developing a fully vectorized CG implementation (CGV) for the CYBER 205 was principally devoted to producing an efficient fast Poisson solver for the preconditioning. Relying on past experience we implemented a matrix decomposition algorithm based on the use of an efficient FFT. Such an implementation requires a highly vectorized multiple FFT subroutine which we were fortunate to obtain from M. Kascic of Control Data Corporation. Using this subroutine (RAVEL) and the algorithm described in [WI], we produced an extremely fast FPS which we dubbed PSOLVE.

The remaining essential CGV processes involved use of a system vector routine for implementing dot products together with a few arithmetic vector operations.

6. Numerical Results

Figure 4 depicts the results of timing tests with the three MG routines. Each was run using B - RBRBR relaxation, one W-cycle and $NX = NY = NZ = 32$. The timings are in milliseconds per cycle categorized in Figure 4 according to: the processes of relaxation (RELAX); calculation of the residual and transfer to the coarser mesh (RESCAL); interpolation of the correction to the finer mesh and addition to the approximation (INTAD); the set-up time for each call to the multigrid subroutine

(Repeated Set-up); the total time for repeated calls for the first and subsequent multigrid cycles (Total: 1st/2nd); and the set-up costs that occur only once per run (One-time Set-up).

Note that MG1 and MG2 have substantial one-time set-up costs. This is due to the need to perform scalar operations to compute full grid length pointers. However, when these costs can be ignored, such as for time-dependent applications using many time steps, MGV2 is clearly the most efficient of the three versions.

Note also that, ignoring one-time set-up, the first MGV2 cycle takes about 57 msec. and subsequent ones take 44 msec. These times are used to scale the performance graphs later in this paper.

The CGV iteration times are given in Figure 5 in milliseconds, categorized according to: one fast Poisson solver (PSOLVE); the first and subsequent CG iterations, excluding calls to PSOLVE (CG: 1st/2nd); the total time for repeated calls for the first and subsequent CGV steps (Total: 1st/2nd); and the set-up times in PSOLVE that occur one per run (One-time Set-up). Note the similarity in total time per step of MGV and CGV.

To test the performance of MGV2 and CGV over a range of decreasingly "Poisson-like" problems, we used the same driver for each listed in Appendix B with the initial guess UIN identically zero, the exact solution $U_{EX}(X,Y,Z) = X + Y + Z$, and the operator coefficient function given by

$$(6.1) \quad ACF(X, Y, Z) = 1.0 - EPS * RANF().$$

The values 0, 0.1, 0.2, 0.5 and 1 were assigned to EPS with the results summarized in Figure 6 for MGV2 and Figure 7 for CGV. These are plots of the log base 10 of the Euclidean norm of the residual error versus execution time. Note that the performance of MGV2 is very consistent for mild coefficient variations ($EPS = 0, 0.1, 0.2$ and 0.5) and that it degrades "gracefully" under stronger variations ($EPS = 1.0$). Since CGV uses a Poisson ($EPS = 0$) solver as a preconditioner, it is expectedly very sensitive to EPS. Note that, even for the near Poisson-type problem treated here ($EPS = 0.1$), CGV is slower than MGV2 for the first few steps. This is quite typical of the numerical experiences we have had with these methods.

7. Conclusions

The performance characteristics of MGV2 and CGV are quite similar for the Poisson-like problems we have in mind. CGV is slightly faster, but a few simple improvements in MGV2 should eliminate this difference. The asymptotic rates are also quite similar. The main differences besides robustness are therefore start-up time and initial convergence rates. For static problems, these should nearly balance so again there is little difference between these two codes. For time-dependent problems, amortization of the start-up time favors MGV2. Indeed, if a few algorithm steps are sufficient to resolve the spatial accuracy, then MGV2 may be substantially more efficient than CGV after many time steps.

The acid test for comparing these codes will involve their use in the fire model code. Tests of this type are currently underway.

A third method that may be more efficient than either of the two suggests itself. Specifically, a fully vectorized Poisson solver based on multigrid can be used in place of RAVEL. If the projections from the work in [BA] prove to be correct, then the PSOLVE time may be reduced significantly enough to make CGV the method of choice here.

REFERENCES

- [RE] Rehm, R.G. and Baum, H.R., "The equations of motion for thermally driven, buoyant flows", J. Res. Nat. Bur. Std. 83 (1978), pp. 297-308.
- [MC] McCormick, S. and Ruge, J., "Multigrid methods for variational problems", SIAM J. Numer. Anal. 19 (1982), 924-929.
- [BA] Barkai, D., "Vectorized multigrid Poisson solver for the CDC CYBER 205", this proceedings.
- [BR] Brandt, A., "Multilevel adaptive solutions to boundary value problems", Math. Comp. 31 (1977), pp. 333-390.
- [HO] Holland, W., McCormick, S. and Ruge, J., "Unigrid methods for boundary value problems on nonrectangular domains", J.Comp. Physics 48 (1982), pp. 412-422.
- [LE] Lewis, J. G. and Rehm, R. G., "The numerical solution of a nonseparable elliptic partial differential equation by preconditioned conjugate gradients", J. Res. Nat. Bur. Std. 85 (1980), pp. 367-390.
- [WI] Wilhelmson, R. B. and Ericksen, J. H., "Direct solution of Poisson's equation in three dimensions", J. Comp. Physics 25 (1977), pp. 319-331.

APPENDIX A*

The CYBER 205

The most significant difference between a traditional, serial computer and a vector processor is the ability of the latter to produce a whole array ("vector") of results upon issuing a single hardware instruction. The input to such a vector-instruction may be one or two vectors, one or two elements ("scalars"), or a combination of the above. The instructions fall into two main categories - those that perform floating-point arithmetic (including square root, sum, dot product, etc., as well as the basic operations), and those which may be collectively called "data-motion" instructions. These may be used, for example, to "gather" elements from one array into another using an arbitrary "index list"; to "compress" or "expand" an array; to "merge" two arrays into one (with arbitrary "inter-leaving" patterns); etc.

The need for vector data-motion instructions becomes apparent when one considers the definition of a vector on a CDC CYBER 205. A vector is a set (array) of elements occupying consecutive locations in memory. It means, by the way, that a vector may be represented in FORTRAN by a multi-dimensional array, i.e., a two- or three-dimensional array may be used in computations as a single vector. The reason for this vector definition is that when performing vector operations on the CDC CYBER 205 the input elements are streamed directly from memory to the vector pipes and the output is streamed directly back into memory without any intermediate registers.

The timing formula for completing a vector instruction contains two components. One is fixed, i.e., independent of the number of elements to be computed and is called "start-up" time. In fact, it amounts to start-up and shut-down: it involves fetching the pointers to the input and output streams, aligning the arrays so as to eliminate bank conflicts and getting the first pair of operands to the functional unit (the pipeline) and the last one back to memory. Typical time for the "start-up" component is 1 microsecond, or about 50 cycles (clock periods). The

*This appendix was written by Dr. David Barkai of CDC's Center for Advanced Vector Technology.

other component of the timing formula is the "stream-time" which is proportional to the number of elements in the vector. The result rate for a 2-pipe CDC CYBER 205 for an add or multiply is 2 results per cycle. It is apparent now that in order to offset the "wasted" cycles of start-up times it is beneficial to work with longer vectors. The system is better utilized if a single operation is performed on a long vector rather than several operations to compute the same number of results. Given a vector length N one can evaluate the efficiency of the computation as the ratio between the number of cycles used to produce useful results and the total number of cycles the instruction has taken, i.e., $(N/2)/((N/2) + 50)$. The maximum vector length the CDC CYBER 205 hardware allows is 65,535 elements. The start-up time becomes quite negligible long before that.

The vector "arguments" for vector instructions are inserted through a construct called Descriptor. It is a quantity occupying 64 bits which fully describes vectors through two integer values: one is the virtual address of the starting location of the vector and the other is the number of elements, or the length, of the vector. An element may be a bit, a byte, a half-word (32 bits) or a word (64 bits) depending on the instruction and the argument within the instruction. The CDC CYBER 205 FORTRAN provides the ability to declare variables of "type" Descriptor and Bit, as well as extensions for assigning Descriptors to arrays and syntax for coding vector instructions without such an explicit association. Bit arrays occupy exactly one bit per element, since the CDC CYBER 205 is bit-addressible. Bit vectors are used for creating a "mapping" between an array containing numerical values and a subset of it. A bit vector may be used to control a vector floating-point operation (whence the term "control-vector" which is commonly used for a bit vector) as follows: Take, for example, an add operation. All the elements of the two input arrays are summed, but only those result-elements where the corresponding element of the control vector is 1 will be stored into the results vector. The other elements will not be modified. Alternatively, one may specify storing on zeros in the control vector and discarding results corresponding to a 1. (The operation is sometimes referred to as a controlled store.)

Another common use of bit vectors is associated with some of the data-motion instructions. Two examples will be given here: the "compress" instruction is used to create a vector which is a subset of another vector. This operation has two input descriptors -- one points to a numeric vector, the other to a bit vector. Whenever a 1 is encountered in the bit vector, the corresponding numeric element is moved to the next location of the output vector, i.e., the input array is "compressed" (the reverse process may be accomplished with an "expand" instruction). A single bit vector may also be used to "merge" two numeric vectors into one. The bit vector is scanned and when a 1 is encountered the next element of the first input vector is put into the next location of the output vector. When a zero is found in the bit vector, the next element of the second input vector is moved into the next location of the output vector. The timing for both these instructions is dictated by the total length of the bit vector. The result rate is the same as that of vector arithmetic, i.e., on a two-pipe CYBER 205 it is two elements per cycle (whether they are moved or not). It will be noted here that there are vector instructions for creating repeated but patterns at a rate of 16 bits per cycle.

Two other vector instructions useful for processing noncontiguous data are commonly referred to as "GATHER" and "SCATTER". They have two input streams: one is a data array, the other is an index list. The "GATHER" instruction steps through the index list, picks up the data element pointed to by the current index and puts it into the next location of the output stream. For example, consider the following:

Data array: 20, 19, 18, ..., 2, 1, 0

Index list: 3, 1, 10, 10, 9, 5, 6, 6, 6 .

Execution of the GATHER instruction the produces the array:

18, 20, 11, 11, 12, 16, 15, 15, 15 .

The SCATTER instruction uses the index list to point to the location in the output array where the next data element is to go. The timing of these instructions is dictated by the length of the index list with one result per 1.25 clock periods (i.e., per 25 nanoseconds) regardless of the number of pipes.

APPENDIX B

MGV CODE


```

PROGRAM DRVMGV(INPUT,OUTPUT,TAPE6=OUTPUT)
C
C TEST DRIVER FOR THE MGV ROUTINE ON THE CYBER 205
C
C THE MESH SIZE IS NX BY NY BY NZ
C
PARAMETER (NX=32,NY=32,NZ=32, NXYZ=NX*NY*NZ, NTH=(NXYZ+1)/2,
1 NG=5)
REAL U(NX,NY,NZ),D1(NX,NY,NZ),D2(NX,NY,NZ),D3(NX,NY,NZ),
1 D4(NX,NY,NZ),R(NX,NY,NZ)
COMMON /TIMING/ CP5(10,5)
C
C THE FUNCTION UEX DEFINES THE SOLUTION OF THE LINEAR SYSTEM.
C THE FUNCTION ACF(X,Y,Z) DEFINES THE COEFFICIENT FUNCTION FOR
C THE SELF ADJOINT FORM OF THE ELLIPTIC EQUATION.
C THE FUNCTION UIN(X,Y,Z) DEFINES THE INITIAL GUESS.
C THE RIGHT SIDE ARRAY IS DEFINED SO THAT UEX IS THE EXACT SOLUTION.
C THE ARRAY D1 CONTAINS THE DIAGONAL OF THE FINITE DIFFERENCE MATRIX.
C THE ARRAY D2 CONTAINS THE SUPER DIAGONAL DETERMINED BY THE (I+1,J,K)
C NEIGHBOR, D3 BY (I,J+1,K), AND D4 BY (I,J,K+1).
C
ACF(X,Y,Z)=1.
UEX(X,Y,Z)=X+Y+Z
UIN(X,Y,Z)=0.
C
C INITIALIZE PARAMETERS
C
IGAMA=2
NCYC=2
C
C SET UP THE INITIAL GUESS FOR THE SOLUTION
C
DX=1./FLOAT(NX)
DY=1./FLOAT(NY)
DZ=1./FLOAT(NZ)
DO 50 I=1,NX
X=(I-.5)/FLOAT(NX)
DO 50 J=1,NY
Y=(J-.5)/FLOAT(NY)
DO 50 K=1,NZ
Z=(K-.5)/FLOAT(NZ)
U(I,J,K)=UIN(X,Y,Z)
50 CONTINUE
C
C SET THE RIGHT SIDE ARRAY SO THAT UEX IS THE EXACT SOLUTION OF
C DIFFERENCE EQUATIONS
C
DO 120 I=1,NX
X=(I-.5)/FLOAT(NX)
DO 120 J=1,NY
Y=(J-.5)/FLOAT(NY)
DO 120 K=1,NZ
Z=(K-.5)/FLOAT(NZ)
R(I,J,K)=0.
D1(I,J,K)=0.
IF(I.LT. NX) THEN
D2(I,J,K)=ACF(X+.5*DX,Y,Z)/DX**2

```

```

R(I,J,K)=R(I,J,K)+D2(I,J,K)*(UEX(X+DX,Y,Z)-UEX(X,Y,Z))
ELSE
D2(I,J,K)=0.
END IF
IF(I.GT. 1) THEN
R(I,J,K)=R(I,J,K)-D2(I-1,J,K)*(UEX(X,Y,Z)-UEX(X-DX,Y,Z))
D1(I,J,K)=D1(I,J,K)-D2(I-1,J,K)
END IF
IF(J.LT. NY) THEN
D3(I,J,K)=ACF(X,Y+.5*DY,Z)/DY**2
R(I,J,K)=R(I,J,K)+D3(I,J,K)*(UEX(X,Y+DY,Z)-UEX(X,Y,Z))
ELSE
D3(I,J,K)=0.
END IF
IF(J.GT. 1) THEN
R(I,J,K)=R(I,J,K)-D3(I,J-1,K)*(UEX(X,Y,Z)-UEX(X,Y-DY,Z))
D1(I,J,K)=D1(I,J,K)-D3(I,J-1,K)
END IF
IF(K.LT. NZ) THEN
D4(I,J,K)=ACF(X,Y,Z+.5*DZ)/DZ**2
R(I,J,K)=R(I,J,K)+D4(I,J,K)*(UEX(X,Y,Z+DZ)-UEX(X,Y,Z))
ELSE
D4(I,J,K)=0.
END IF
IF(K.GT. 1) THEN
R(I,J,K)=R(I,J,K)-D4(I,J,K-1)*(UEX(X,Y,Z)-UEX(X,Y,Z-DZ))
D1(I,J,K)=D1(I,J,K)-D4(I,J,K-1)
END IF
D1(I,J,K)=D1(I,J,K)-D2(I,J,K)-D3(I,J,K)-D4(I,J,K)
120 CONTINUE
C
C ORTHOGONALIZE R.H.S. FROM THE SPACE OF CONSTANT VECTORS.
C
S=Q8SSUM(R(1,1,1;NXYZ))
S=-S/FLOAT(NXYZ)
R(1,1,1;NXYZ)=R(1,1,1;NXYZ)+S
C
C CALL THE SOLVER
C
DO 300 ICALL=0,1
CP5(1,1,50)=0.
CPUL=SECOND()
CALL MGV(U,R,D1,D2,D3,D4,NXYZ,NX,NY,NZ,NG,
1 IGAMA,NCYC,ICALL)
CPUL=SECOND()-CPUL
CALL RSDUAL(NG,RES)
C
C COMPUTE THE ERROR
C
UERMAX=0.
USHIFT=U(1,1,1)-UEX(.5*DX,.5*DY,.5*DZ)
DO 220 I=1,NX
DO 220 J=1,NY
DO 220 K=1,NZ
X=(I-.5)/FLOAT(NX)
Y=(J-.5)/FLOAT(NY)

```

```

      Z=(K-.5)/FLOAT(NZ)
      UERIJ=U(I,J,K)-UEX(X,Y,Z) - USHIFT
      UERMAX=AMAX1(UERMAX,ABS(UERIJ))
220    CONTINUE
      WRITE (6,250) UERMAX,RES,CPU1
250    FORMAT(/'---MAX ERROR=',1PE11.4,' RES=',E11.4,' CPU=',E9.2///)
      DO 253 M=1,5
      DO 253 L=1,NG
253    CP5(NG+1,M)=CP5(NG+1,M)+CP5(L,M)
      WRITE (6,255) (L,(CP5(L,M),M=1,4),L=1,NG+1)
255    FORMAT(1X,'****TIMING****'//1X,'LEVEL(6=TOTAL)      RELAX'
+ '      RESCAL      INTAD      MISC'/(5X,I3,8X,4E12.3))
300    CONTINUE
      STOP
      END

```

```

C      SUBROUTINE MGV(U,F,D1,D2,D3,D4,NXYZ,NX,NY,NZ,
+NG,IGAMA,NCYC,ICALL)

```

```

C
C
C VECTORIZED MG VERSION #1 FOR 3-D SYMM. CELL CENTERED
C DIFFUSION EQ. W/ NEUMANN BOUNDARY CONDS.

```

```

C
C I/O VARIABLES:

```

```

C
C U      PRESENT APPROX.
C F      R.H.S.
C D1-D4   MAIN AND X,Y,Z DIAGONALS,RESP.
C NX,NY,NZ # CELLS IN RESP. DIRECTION.
C NG      # LEVELS. (!!! NX,NY, AND NZ MUST EACH BE A MULTIPLE
C          OF 2 TO THE POWER NG !!!)
C IGAMA   FOR MG CYCLING CONTROL. (1 FOR V-CYCLE, 2 FOR W-CYCLE)
C NCYC    # MG CYCLES.
C ICALL   THIS SPECIFIES HOW MANY TIMES MG HAS BEEN CALLED FOR
C          THIS RUN.
C MSH     THIS SHOULD BE SET IN THE PARAMETER STATEMENT TO AT
C          LEAST 8*NX*NY*NZ/7.
C MSHC    THIS SHOULD BE SET IN THE PARAMETER STATEMENT TO AT
C          LEAST NX*NY*NZ/7.

```

```

C
C
C BIT BMESH
C PARAMETER (MSH=38000,MSHC=5000)
C DIMENSION U(NX,NY,NZ),F(NX,NY,NZ),D1(NX,NY,NZ),D2(NX,NY,NZ),
+ D3(NX,NY,NZ),D4(NX,NY,NZ)
C COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3)
C 1,NL(10),NS(10),NXK(10),NYK(10),NZK(10),BMESH(MSH,2),RTMP(MSH),NGG
C DIMENSION IC(10),NREL(3)
C COMMON /TIMING/ CP5(10,5)

```

```

C
C CP2=SECOND()
C CP1=SECOND()
C NGG=NG
C IF(ICALL.GT.0)GO TO 90

```

```

C
C SET UP POINTERS FOR P,R,C, & T TO START AND LENGTH OF LEVEL K

```

```

C
  K=NG
  NL(K)=NXYZ
  NS(K)=1
  NXK(K)=NX
  NYK(K)=NY
  NZK(K)=NZ
80  IF(K.EQ.1) GO TO 85
      K=K-1
      NL(K)=NL(K+1)/8
      NS(K)=NS(K+1)+NL(K+1)
      NXK(K)=NXK(K+1)/2
      NYK(K)=NYK(K+1)/2
      NZK(K)=NZK(K+1)/2
      GO TO 80

C
C  DEFINE # RELAXATIONS FROM FINE TO COARSE, COARSE TO FINE, & ON
C  THE COARSEST LEVEL
C
85  NREL(1)=1
      NREL(2)=3
      NREL(3)=5

C
C  SET UP POINTERS FOR VECTOR ARITHMETIC
C
      CALL INDXS
90  CONTINUE
      P(1,NXYZ)=U(1,1,1,NXYZ)
      R(1,NXYZ)=F(1,1,1,NXYZ)
      C(1,NXYZ)=D1(1,1,1,NXYZ)
      T(1,1,NXYZ)=D2(1,1,1,NXYZ)
      T(1,2,NXYZ)=D3(1,1,1,NXYZ)
      T(1,3,NXYZ)=D4(1,1,1,NXYZ)

C
C  SET UP COARSE LEVEL MATRICES
C
      CALL CGCOEF

C
C  OUTPUT THE INITIAL ERROR
C
      IF(ICALL.EQ.0) CALL RSDUAL(NG,RES)
      NR=NREL(1)
      K=NG
      IC(NG)=IGAMA-NCYC
      CP5(1,4)=SECOND()-CP1

C
C  RELAX ON LEVEL K
C
10  CP1=SECOND()
      CALL RELAX(K,NR,RES)
      CP5(K,1)=CP5(K,1)+SECOND()-CP1
      IF(IC(K).EQ.IGAMA.OR.K.EQ.1) GO TO 30

C
C  COMPUTE RESIDUAL AND TRANSFER TO COARSER LEVEL
C
      CP1=SECOND()

```

```

      CALL RESCAL(K,K-1)
      CP5(K,2)=CP5(K,2)+SECOND()-CP1
C
C      INITAILIZE COARSE LEVEL APPROX.
C
      NO=NS(K-1)
      N1=NL(K-1)
      P(NO,N1)=0.
      K=K-1
      NR=NREL(1)
      IF(K.EQ.1) NR=NREL(3)
      IC(K)=0
      GO TO 10
C
30 IF(K.LT.NG) GO TO 50
   U(1,1,1,NXYZ)=P(1,NXYZ)
   CP5(1,5)=SECOND()-CP2
   RETURN
C
C      INTERPOLATE CORRECTION TO FINER LEVEL AND ADD TO APPROX.
C
50   CP1=SECOND()
      CALL INTAD(K,K+1)
      CP5(K,3)=CP5(K,3)+SECOND()-CP1
      K=K+1
      IC(K)=IC(K)+1
      NR=NREL(2)
      IF(IC(K).LT.IGAMA) NR=NREL(1)+NREL(2)
      GO TO 10
C
      END
C
      SUBROUTINE RELAX(LEV,NR,RES)
      BIT BMESH
      PARAMETER (MSH=38000,MSHC=5000)
      COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3)
      1,NL(10),NS(10),NXK(10),NYK(10),NZK(10),BMESH(MSH,2),RTMP(MSH),NG
C
C      RED BLACK RELAXATION VIA JACOBI WITH BIT VECTORS FOR CONTROL STORE
C
      NX=N XK(LEV)
      NY=NYK(LEV)
      NZ=NZK(LEV)
      NXY=NX*NY
      NXYZ=NXY*NZ
      LB=NS(LEV)
      N1=NXYZ-1
      N2=NXYZ-NX
      N3=NXYZ-NXY
C
      DO 1000 IT=1,NR
      LS=1
      LF=2
      IF(NR.GT.3) GO TO 101

```

```

      IF (NR.EQ.1) LS=2
      IF (IT.EQ.3) LF=1
101  DO 100 L=LS,LF
      RTMP(1,NXYZ)=R(LB,NXYZ)
      RTMP(1,N1)=RTMP(1,N1)
1    -T(LB,1,N1)*P(LB+1,N1)
      RTMP(2,N1)=RTMP(2,N1)
1    -T(LB,1,N1)*P(LB,N1)
      RTMP(1,N2)=RTMP(1,N2)
1    -T(LB,2,N2)*P(LB+NX,N2)
      RTMP(1+NX,N2)=RTMP(1+NX,N2)
1    -T(LB,2,N2)*P(LB,N2)
      RTMP(1,N3)=RTMP(1,N3)
1    -T(LB,3,N3)*P(LB+NXY,N3)
      RTMP(1+NXY,N3)=RTMP(1+NXY,N3)
1    -T(LB,3,N3)*P(LB,N3)
      WHERE(BMESH(LB,L,NXYZ)) P(LB,NXYZ)=RTMP(1,NXYZ)/C(LB,NXYZ)
100  CONTINUE
1000 CONTINUE
C
      RETURN
      END
C-----
      SUBROUTINE RESCAL(LEV,LEVC)
      BIT BMESH
      PARAMETER (MSH=38000,MSHC=5000)
      COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3)
1,NL(10),NS(10),NXK(10),NYK(10),NZK(10),BMESH(MSH,2),RTMP(MSH),NG
      COMMON /PNTRS/ IRESR(MSHC),MP(10,3,4),LP(10,4)
C
C
C
C  SIMPLE AVERAGE TRANSFER OF RESIDUALS FROM LEV TO LEVC
      NST=NS(LEVC)-NL(NG)
      CALL RESID(LEV)
      N0=NS(LEVC)
      N1=NL(LEVC)
      M0=NS(LEV)
      M1=NL(LEV)
      MX=NXK(LEV)
      MXY=MX*NYK(LEV)
      R(N0,N1)=Q8VGATHR(RTMP(M0,M1),IRESR(NST,N1),R(N0,N1))
      M0=M0+MX+1
      P(N0,N1)=Q8VGATHR(RTMP(M0,M1),IRESR(NST,N1),P(N0,N1))
      M0=M0+MXY-MX
      R(N0,N1)=R(N0,N1)+P(N0,N1)
      RTMP(N0,N1)=Q8VGATHR(RTMP(M0,M1),IRESR(NST,N1),RTMP(N0,N1))
      M0=M0+MX-1
      P(N0,N1)=Q8VGATHR(RTMP(M0,M1),IRESR(NST,N1),P(N0,N1))
      P(N0,N1)=P(N0,N1)+RTMP(N0,N1)
      R(N0,N1)=R(N0,N1)+P(N0,N1)
      R(N0,N1)=0.125*R(N0,N1)
C
      RETURN
      END
C-----

```

100

C

C.

C
C
C
C

C

```

N0=NS(NG-1)
C(N0,MSHC)=0.
DO 1000 K=NG,2,-1
N0=NS(K-1)
NST=N0-NL(NG)
NL=NL(K-1)
NI(1)=N0-1
NI(2)=N0-NXK(K-1)
NI(3)=N0-NXK(K-1)*NYK(K-1)
DO 400 J=1,3
DO 100 I=1,4
LI=LP(K,I)
MI=MP(K,I,J)

```

```

100  RTMP(LI,N1)=Q8VGATHR(T(MI,J,M1),IRESR(NST,N1),RTMP(LI,N1))
      DO 200 I=1,2
      LI=LP(K,I)
      LIP=LP(K,2*I)
200  RTMP(LI,N1)=RTMP(LI,N1)+RTMP(LIP,N1)
      LI=LP(K,1)
      LIP=LP(K,2)
      T(N0,J,N1)=RTMP(LI,N1)+RTMP(LIP,N1)
      T(N0,J,N1)=0.0625*T(N0,J,N1)
      C(N0,N1)=C(N0,N1)-T(N0,J,N1)
      NII=NI(J)
400  C(N0,N1)=C(N0,N1)-T(NII,J,N1)
1000 CONTINUE
C

```

```

      RETURN
      END

```

```

C-----
      SUBROUTINE RSDUAL(LEV,RES)
      BIT BMESH
      PARAMETER (MSH=38000,MSHC=5000)
      COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3)
      1,NL(10),NS(10),NXK(10),NYK(10),NZK(10),BMESH(MSH,2),RTMP(MSH),NG

```

```

C
C
C COMPUTE L2 NORM OF LEV RESIDUAL AND OUTPUT
C

```

```

      CALL RESID(LEV)
      N0=NS(LEV)
      N1=NL(LEV)
      RES=Q8SDOT(RTMP(N0,N1),RTMP(N0,N1))
      RES=SQRT(RES*(8.** (LEV-NG)))

```

```

C
      IF(LEV.EQ.NG) WRITE(6,10) RES
      IF(LEV.EQ.NG-1) WRITE(6,100) RES
      IF(LEV.EQ.NG-2) WRITE(6,200) RES
      IF(LEV.EQ.NG-3) WRITE(6,300) RES
      IF(LEV.EQ.NG-4) WRITE(6,400) RES
100  FORMAT(T8,'RES=',E10.3)
200  FORMAT(T16,'RES=',E10.3)
300  FORMAT(T24,'RES=',E10.3)
400  FORMAT(T32,'RES=',E10.3)
10  FORMAT(T2,'RES=',E10.3)

```

```

C
      RETURN
      END

```

```

C-----
      SUBROUTINE RESID(LEV)
      BIT BMESH

```

```

C
C
C COMPUTATION OF LEV RESIDUAL
C

```

```

      PARAMETER (MSH=38000,MSHC=5000)
      COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3),NL(10),
1  NS(10),NXK(10),NYK(10),NZK(10),BMESH(MSH,2),
2  RTMP(MSH),NG

```

C

```

NX=NKK(LEV)
NY=NYK(LEV)
NZ=NZK(LEV)
NXY=NX*NY
NXYZ=NXY*NZ
LB=NS(LEV)
N1=NXYZ-1
N2=NXYZ-NX
N3=NXYZ-NXY

```

C

```

RTMP(1,NXYZ)=R(LB,NXYZ)
RTMP(1,N1)=RTMP(1,N1)
1 -T(LB,1,N1)*P(LB+1,N1)
RTMP(2,N1)=RTMP(2,N1)
1 -T(LB,1,N1)*P(LB,N1)
RTMP(1,N2)=RTMP(1,N2)
1 -T(LB,2,N2)*P(LB+NX,N2)
RTMP(1+NX,N2)=RTMP(1+NX,N2)
1 -T(LB,2,N2)*P(LB,N2)
RTMP(1,N3)=RTMP(1,N3)
1 -T(LB,3,N3)*P(LB+NXY,N3)
RTMP(1+NXY,N3)=RTMP(1+NXY,N3)
1 -T(LB,3,N3)*P(LB,N3)
RTMP(LB,NXYZ)=RTMP(1,NXYZ)-C(LB,NXYZ)*P(LB,NXYZ)

```

C

```

RETURN
END

```

C

```

-----
SUBROUTINE INDXS
BIT BMESH
PARAMETER (MSH=38000,MSHC=5000)
COMMON /MGVAR/ P(MSH),R(MSH),C(MSH),T(MSH,3)
1,NL(10),NS(10),NKK(10),NYK(10),NZK(10),BMESH(MSH,2),RTMP(MSH),NG
COMMON /PNTRS/ IRESR(MSHC),MP(10,4,3),LP(10,4)

```

C

C

```

C COMPUTE INDEXED ARRAYS AND BIT VECTORS FOR VECTOR ARITHMETIC
C IQ=0
C DO 100 K=NG,2,-1

```

C

C

C

C

```

C IRESR POINTS TO THE LEVEL K CELL THAT SITS IN THE BOTTOM SOUTH
C WEST CORNER OF THE LEVEL K-1 CELL

```

```

M=1
MXC=NKK(K-1)
MYC=NYK(K-1)
MZC=NZK(K-1)
MX=NKK(K)
MXY=MX*NYK(K)
DO 30 KK=1,MZC
DO 20 JJ=1,MYC
DO 10 II=1,MXC
IQ=IQ+1
IRESR(IQ)=M
10 M=M+2

```

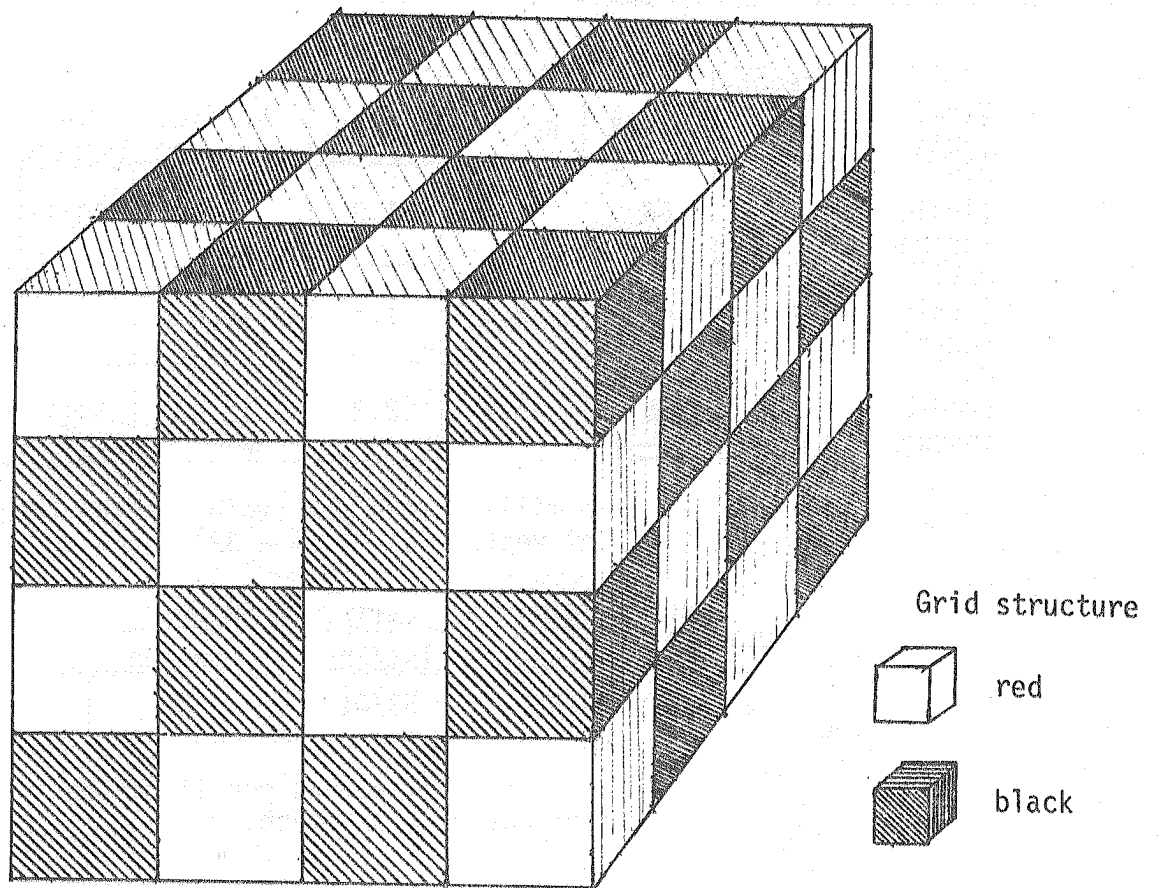


Figure 1

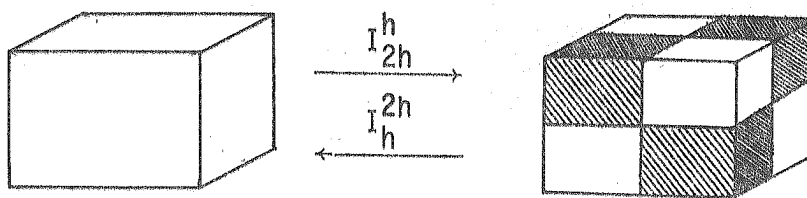


Figure 2. Integral Transfers



Figure 3. IRESR

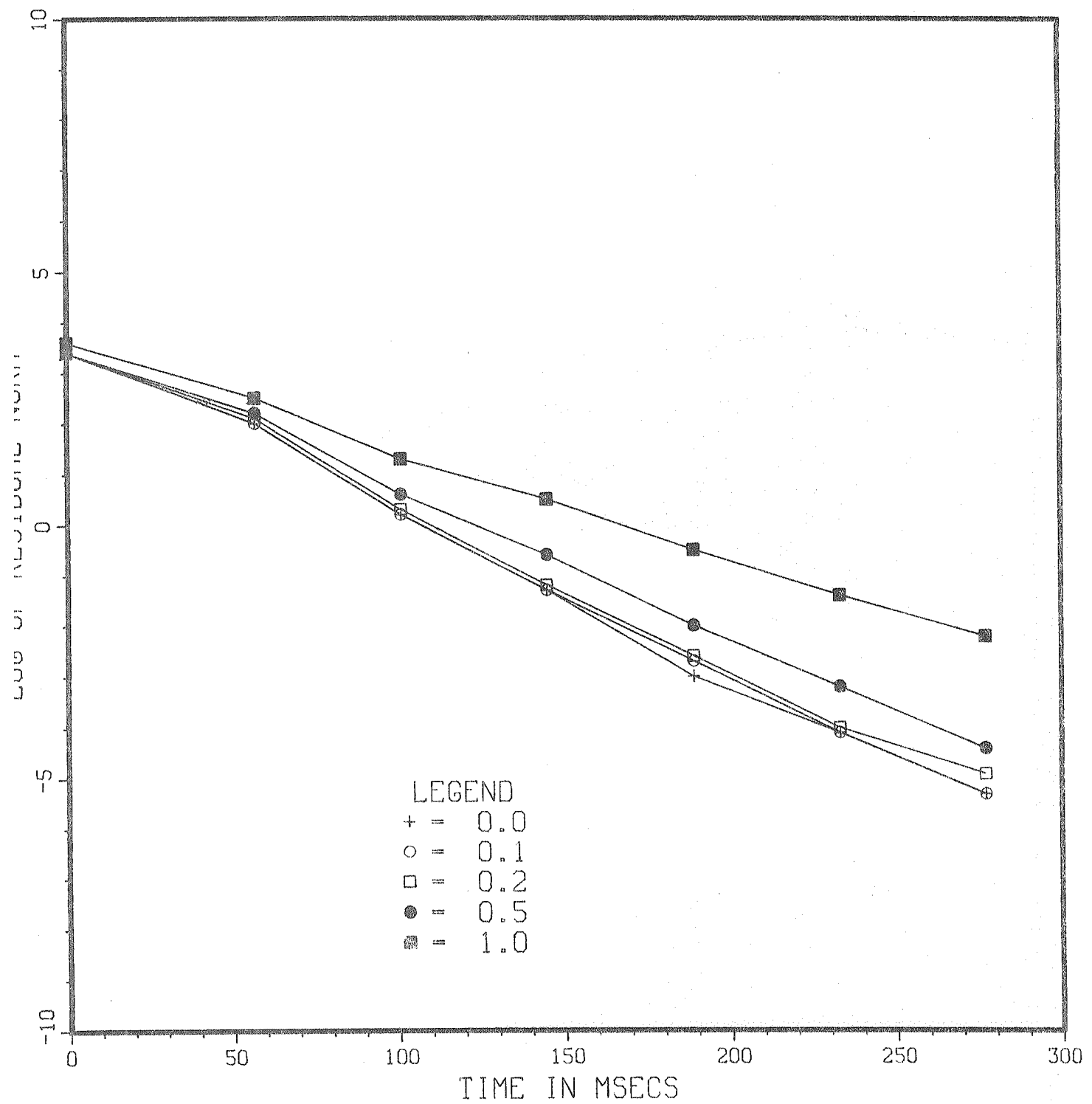


Figure 7. Plot of error versus time for CGV.